

MoMo SMS Data Processing Application

Database Design Document

Week 2 Submission

Team: Team 1

Course: Enterprise Web Development

Trimester: May 2026

Team Members:

1. Aduk Mathiang Ngut Nyang
2. Ntwali Beni David
3. Bol David Garang Dau
4. Ojudun Ayomide Oluwatimilehin

1. Introduction

This document presents the database design for the MoMo SMS Data Processing Application. The application reads MTN MoMo SMS messages exported as XML, parses them into structured records, and stores them in a relational database that can later support a dashboard for transaction analysis. Before designing anything, our team went through the full XML dataset and looked at every type of message that appears, so we could understand exactly what fields are present, what is sometimes missing, and what is unique to certain transaction types.

Across the dataset, the fields that appear in every single transaction are the date and time. The amount, balance, and counterparty information appear in most but not all transaction types, while fields like the fee and the transaction ID only show up in specific types. This observation shaped a lot of our design decisions, especially around which columns we allow to be NULL and which entity a piece of information actually belongs to.

Transaction Types Identified

From the XML, we identified the following distinct transaction types:

- Incoming Money
- Payments (to a person, to a code, Airtime Purchase, Cash Power)
- Bank Deposit
- Transfer to Mobile
- Merchant Payment
- Cash Withdrawal
- OTP / system messages

Field Availability Observations

After comparing the message bodies of each transaction type, we noticed the following patterns:

- The amount in RWF and the new balance appear in Incoming Money, Payments, Bank Deposit, Transfer, and Cash Withdrawal — but not in OTP or system messages.

- Date and time appear in all messages without exception.
- Transaction ID does not appear in Bank Deposit, Transfer, or OTP messages.
- Fee charges only apply to Payments, Transfers, and Cash Withdrawals.
- Bank Deposits do not include a counterparty, so we do not know who sent the money.
- Counterparty phone numbers are not provided in Payments or Bank Deposits.
- For Incoming Money, we sometimes also receive a message from the sender (starting with “Message from sender:”), but most of the time the sender does not write a message.

Regarding OTP messages, we decided to put them in their own entity since they do not share most of the details with other financial transactions like amount, balance, fee, or counterparty. Forcing them into the same table as financial transactions would mean almost every column would be NULL for them, which is poor design.

2. Entity Overview

Our database has six entities. Transactions is the central table that stores every money movement. Users hold the people, businesses, and agents involved in those movements. Transaction_Categories classifies the type of each transaction. Transaction_Users is the junction table that links Transactions to Users as a many-to-many relationship, while keeping a role distinction between sender, receiver, or agent. System_Logs preserves the ETL processing history for debugging and traceability, and System_Messages stores OTPs separately because they do not share the financial schema.

Entities and Primary Key Strategy

The choice of primary key was different for each entity, depending on whether a reliable natural key existed in the source data.

Entity	Primary Key	Why we chose it
Transactions	Auto-generated transaction_id	We could have used the transaction date but since two transactions can happen at the same time we decided to use the autogenerated transaction_id.
Users	Auto-generated user_id	Phone numbers cannot be reliably traced to one user, and in many transactions the phone number is not even present. A surrogate key gives us a stable identifier.
Transaction_Categories	Auto-generated category_id	We needed a clean lookup table for the type of transaction. The surrogate key also speeds up JOINS from the central Transactions table.
Transaction_Users	Composite (transaction_id, user_id, role)	This junction table resolves the M:N relationship. The composite key ensures the same user cannot appear twice in the same role for the same transaction.

Entity	Primary Key	Why we chose it
System_Logs	Auto-generated log_id	Each log row is just an event; a surrogate key is the simplest fit.
System_Messages	received_at (timestamp)	OTPs are timestamped events with no other natural identifier; the receive time works as a natural key.

Some entities like Transactions and System_Messages use natural keys while others use auto-increment surrogate keys. We made this decision per-entity instead of forcing one rule everywhere, because we verified the XML data and confirmed where natural uniqueness actually holds.

3. Design Rationale

In this section we explain the key decisions behind the design so that anyone reading the schema later understands the reasoning, not just the result.

Transaction Categories

Transaction_Categories covers all the types we identified: Incoming Money, Payment (to person, code, airtime, cash power, merchant), Bank Deposit, Transfer, and Cash Withdrawal. To make dashboard work easier later on, we added a direction column that specifies whether the money is coming IN or going OUT, and we went a step further by adding a type column that distinguishes Transfer, Payment, Deposit, and Withdrawal. As the primary key we saw it was more effective to auto-generate an ID as new categories occur, and this also helps increase JOIN speed against the Transactions table.

Transactions

Transactions is the central entity in the schema. It stores the financial details: a surrogate transaction_id as the primary key, the timestamp (transaction_date) and human-readable occurred_at, the amount, the fee, the new balance after the transaction, the category it belongs to, an optional sender message, the internal token from the SMS, the external transaction ID where available, and the raw SMS body for traceability. We decided to use DECIMAL for monetary amounts because it is the standard worldwide format for financial calculations and avoids the rounding errors that FLOAT or DOUBLE would introduce.

OTPs in a Separate Entity

OTP messages are kept in their own entity (System_Messages) because they carry no financial information — no amount, no balance, no counterparty, no fee. Mixing them into Transactions would mean nearly every financial column would be NULL for those rows, which violates good normalization practice and makes querying the financial dataset slower and noisier. The table still keeps a user_id foreign key so that, when we can identify the OTP recipient, we can link the message back to the user.

System Logs Linkage

System_Logs records every event that happens during the ETL pipeline — successful parses, parse failures, validation warnings, and so on. Because many of these events are tied to a specific transaction, we added a transaction_id foreign key on this table. The column is nullable, because some log events are global (for example, “batch started”) and do not relate to any single transaction.

User Type Classification

The `user_type` column uses a three-value classification: agent, business, and customer. These are the categories of people that actually appear in our transactions. MoMo agents are tagged as agent, entities written in all capital letters or names ending in “Ltd” are tagged as business, and everyone else is labeled simply as customer. This classification helps when later building dashboards that separate customer-to-customer activity from business activity or agent operations.

NULL Handling for Fees

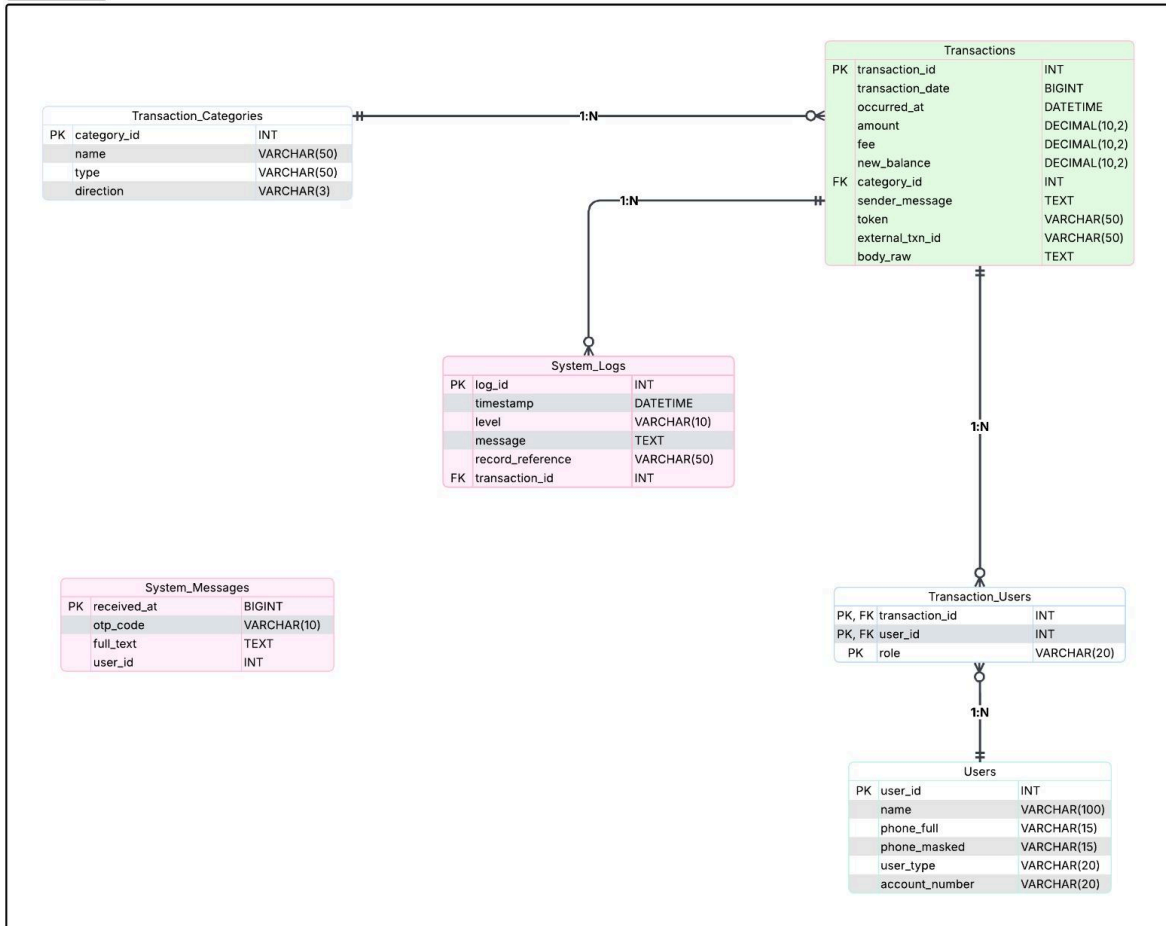
The fee column allows NULL because fees only apply to Transfers, Withdrawals, and some Payments. Other transaction types either do not charge a fee or do not report one. Storing zero in those rows would be misleading because it would suggest that a fee was charged and it happened to be zero, when in reality no fee concept applies at all.

The Many-to-Many Relationship

The relationship between Transactions and Users is many-to-many. A single transaction can involve multiple users — a sender, a receiver, and sometimes an agent — and each user can be involved in many different transactions as time passes. A relational database cannot implement an M:N relationship directly, and that is where `Transaction_Users` comes in. It stores the records of each user involved in a transaction along with their role in that transaction. The composite primary key (`transaction_id`, `user_id`, `role`) ensures the same user can appear only once per role in a given transaction, while still allowing one user to have multiple distinct roles in the same transaction if that ever becomes necessary.

4. Entity Relationship Diagram (ERD)

The ERD below shows the six entities, their attributes with data types, their primary and foreign keys, and the cardinality of each relationship. The data dictionary in section 5 provides the authoritative column-level reference, including the `transaction_id` surrogate key on Transactions, the `transaction_id` foreign key on System_Logs, and the `user_id` foreign key on System_Messages, all of which were aligned across the team during the Week 2 peer review.



Relationship Cardinality Summary

Relationship	Cardinality	Meaning
Transaction_Categories → Transactions	1 : M	One category can classify many transactions; each transaction belongs to exactly one category. Implemented through Transactions.category_id.
Transactions ↔ Users	M : N	Resolved via Transaction_Users. A transaction can have multiple users (sender, receiver, agent), and a user can take part in many transactions.
Transactions → System_Logs	1 : M	One transaction may generate several log entries during ETL. Implemented through System_Logs.transaction_id (nullable, because some log events are global).
Users → System_Messages	1 : M	One user may receive many OTPs over time. Implemented through System_Messages.user_id (nullable,

Relationship	Cardinality	Meaning
		because the recipient is not always identifiable).

5. Data Dictionary

Below is the full data dictionary for every table in the schema. Each column lists its data type, whether it is required, whether it is a key, and what it represents.

Transactions

Column	Type	Key	Null ?	Description
transaction_id	INT AUTO_INCREMENT	PK	NO	Surrogate identifier for each transaction. Used as the foreign-key target by Transaction_Users and System_Logs.
transaction_date	BIGINT	-	NO	Unix timestamp from the source SMS, kept for chronological sorting and reporting.
occurred_at	DATETIME	-	NO	Human-readable form of the transaction time.
amount	DECIMAL(10,2)	-	NO	Transaction amount in RWF.
fee	DECIMAL(10,2)	-	YES	Fee charged. NULL when the transaction type carries no fee concept.
new_balance	DECIMAL(10,2)	-	YES	Account balance after the transaction. Missing for OTP-like rows.
category_id	INT	FK	NO	References Transaction_Categories.category_id.
sender_message	TEXT	-	YES	Optional free text from the sender on Incoming Money.
token	VARCHAR(50)	-	YES	Internal token from the SMS body.
external_txn_id	VARCHAR(50)	-	YES	External transaction ID where the SMS provides one.
body_raw	TEXT	-	NO	Original SMS body, kept for traceability and re-parsing.

Users

Column	Type	Key	Null ?	Description
user_id	INT AUTO_INCREMENT	PK	NO	Surrogate identifier for each distinct user.
name	VARCHAR(100)	-	NO	Name as it appears in the SMS.
phone_full	VARCHAR(15)	-	YES	Full phone number when available.
phone_masked	VARCHAR(15)	-	YES	Masked form (e.g. +25078***567) used in dashboards.
user_type	VARCHAR(20)	-	NO	One of: 'agent', 'business', 'customer'.
account_number	VARCHAR(20)	-	YES	Account number when present in the SMS.

Transaction_Categories

Column	Type	Key	Null ?	Description
category_id	INT AUTO_INCREMENT	PK	NO	Surrogate identifier for each category.
name	VARCHAR(50)	-	NO	Human-readable category name (e.g. 'Incoming Money', 'Cash Power').
type	VARCHAR(50)	-	NO	One of: 'transfer', 'payment', 'deposit', 'withdrawal'.
direction	VARCHAR(3)	-	NO	Either 'IN' or 'OUT' relative to the account.

Transaction_Users (Junction Table)

Column	Type	Key	Null ?	Description
transaction_id	INT	PK, FK	NO	References Transactions.transaction_id.
user_id	INT	PK, FK	NO	References Users.user_id.
role	VARCHAR(20)	PK	NO	One of: 'sender', 'receiver', 'agent'.

System_Logs

Column	Type	Key	Null ?	Description
log_id	INT AUTO_INCREMENT	PK	NO	Surrogate identifier for each log event.
timestamp	DATETIME	-	NO	When the log event occurred.
level	VARCHAR(10)	-	NO	One of: 'INFO', 'WARN', 'ERROR'.
message	TEXT	-	NO	Description of the event.
record_reference	VARCHAR(50)	-	YES	Reference back to the source record (e.g. an SMS token or external ID).
transaction_id	INT	FK	YES	References Transactions.transaction_id when the log event relates to a specific transaction. NULL for global events like 'batch started'.

System_Messages

Column	Type	Key	Null ?	Description
received_at	BIGINT	PK	NO	Unix timestamp when the OTP was received.
otp_code	VARCHAR(10)	-	YES	The OTP code if extracted.
full_text	TEXT	-	NO	Full body of the OTP / system message.
user_id	INT	FK	YES	References Users.user_id when the recipient is known. NULL when the OTP cannot be tied to a specific user.

6. SQL Implementation Notes

The full DDL script is in the repository at database/database_setup.sql. Below we highlight the implementation choices that go beyond a simple translation of the ERD.

Constraints

- FOREIGN KEY constraints connect Transactions to Transaction_Categories, Transaction_Users to both Transactions and Users, and System_Logs / System_Messages back to Transactions and Users, respectively.
- CHECK constraint on Users.user_type to enforce the three allowed values: 'agent', 'business', 'customer'.

- CHECK constraint on Transaction_Categories.direction to allow only 'IN' or 'OUT'.
- CHECK constraint on Transactions.amount to enforce amount ≥ 0 .
- CHECK constraint on Transactions.fee to enforce fee ≥ 0 when not NULL.
- UNIQUE constraint on Users.phone_full so the same phone is not registered twice.

Indexes for Performance

- Index on Transactions.category_id to speed up JOINS to Transaction_Categories.
- Index on Transactions.occurred_at to speed up date-range queries that the dashboard will run often.
- Index on Transaction_Users.user_id for fast lookups of all transactions for a given user.
- Composite index on (transaction_id, role) inside Transaction_Users to speed up role-specific lookups such as 'find all senders'.

Column Comments

Each column in the DDL script includes a COMMENT clause describing what it represents, so that anyone connecting via a SQL client or running SHOW FULL COLUMNS gets in-database documentation immediately, without having to open this file.

7. Security and Accuracy Rules Added

Beyond the default constraints, we added rules to protect the data from common quality and security issues.

Rule	Where it lives	Why it matters
Phone number masking	Users.phone_masked	Dashboards consume phone_masked instead of phone_full, so personally identifiable information is not exposed in the UI layer.
Amount non-negativity	CHECK (amount ≥ 0)	Prevents data entry or parsing bugs from inserting negative amounts that would corrupt totals.
Fee non-negativity	CHECK (fee IS NULL OR fee ≥ 0)	Allows NULL for transaction types with no fee concept, while blocking negative fees.
Direction whitelist	CHECK (direction IN ('IN','OUT'))	Prevents typos like 'in', 'out ', or unrelated values from sneaking into the category table.
User type whitelist	CHECK (user_type IN ('agent','business','customer'))	Forces every user to be classified into one of the three known categories.
Role whitelist on junction	CHECK (role IN ('sender','receiver','agent'))	Stops invalid roles from being attached to a transaction.

Rule	Where it lives	Why it matters
Unique phone number	UNIQUE (phone_full)	Prevents the same person being inserted twice when their phone number is known.
Composite uniqueness on roles	PRIMARY KEY (transaction_id, user_id, role)	Ensures the same user cannot be recorded twice in the same role for the same transaction.
Referential integrity	FOREIGN KEY ... ON DELETE RESTRICT	Prevents accidental deletion of a category or a user that is still referenced by transactions.

8. CRUD Operations Testing

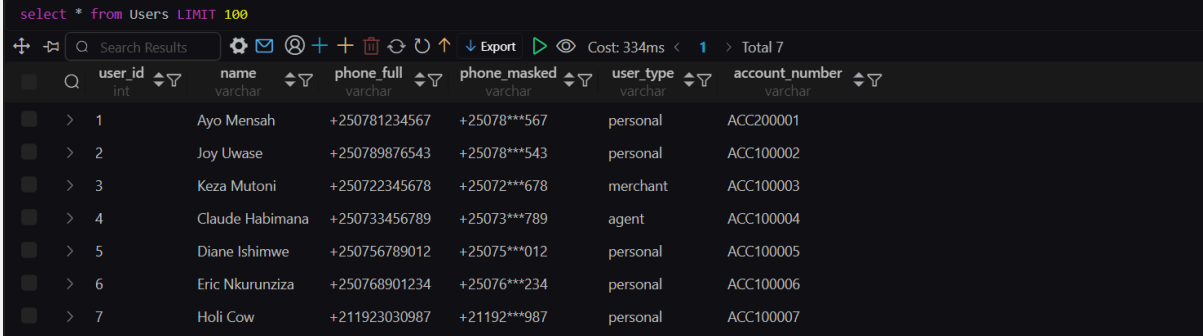
This section demonstrates that the database supports the four core CRUD operations: Create, Read, Update, and Delete. We ran each operation on the Users table and captured the result before and after, so the effect of each query is visible. The Users table was chosen for this test because it is referenced by every other entity in the schema, so a successful CRUD cycle here also validates the constraints that protect the rest of the database.

All operations were executed against the live momo_db database. The screenshots below show the exact SQL we ran, the AffectedRows feedback from the client, and the state of the table afterwards.

Query 1: READ (initial state of the Users table)

We start with a simple SELECT to confirm the table has been seeded with our sample data. At this point the table contains seven users, which matches the records we inserted during the database setup script.

```
select * from Users LIMIT 100;
```



	user_id	name	phone_full	phone_masked	user_type	account_number
	int	varchar	varchar	varchar	varchar	varchar
>	1	Ayo Mensah	+250781234567	+25078***567	personal	ACC200001
>	2	Joy Uwase	+250789876543	+25078***543	personal	ACC100002
>	3	Keza Mutoni	+250722345678	+25072***678	merchant	ACC100003
>	4	Claude Habimana	+250733456789	+25073***789	agent	ACC100004
>	5	Diane Ishimwe	+250756789012	+25075***012	personal	ACC100005
>	6	Eric Nkurunziza	+250768901234	+25076***234	personal	ACC100006
>	7	Holi Cow	+211923030987	+21192***987	personal	ACC100007

Query 2: CREATE (insert a new user)

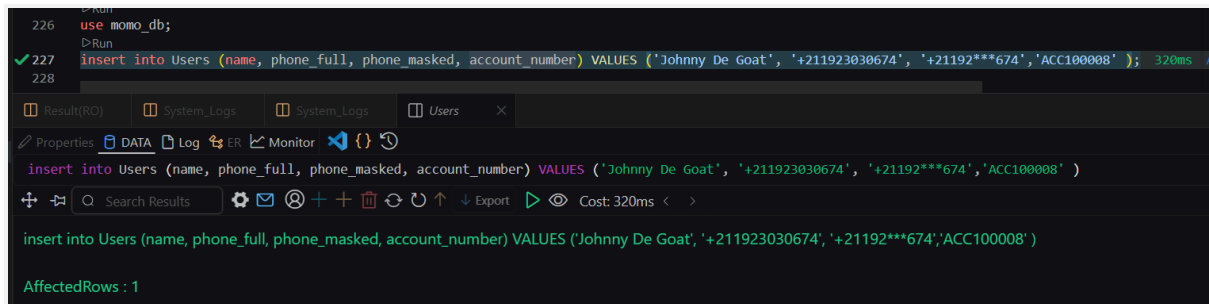
Next, we insert a new user, “Johnny De Goat”, into the Users table. We provide values for name, phone_full, phone_masked, and account_number. The user_id is left out because it is AUTO_INCREMENT and will be generated by the database. The client confirms AffectedRows: 1, which means exactly one row was inserted.

```
insert into Users (name, phone_full, phone_masked, account_number)
VALUES ('Johnny De Goat', '+211923030674', '+21192***674', 'ACC100008');
```

```

226 use momo_db;
227 insert into Users (name, phone_full, phone_masked, account_number) VALUES ('Johnny De Goat', '+211923030674', '+21192***674', 'ACC100008'); 320ms
228

```



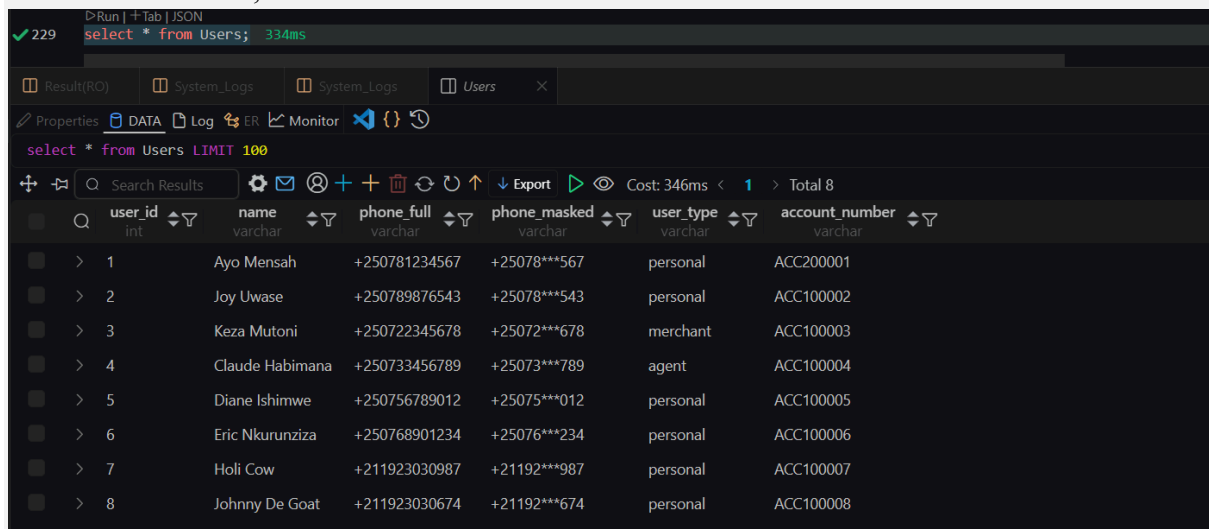
To verify that the new row actually persisted, we ran a SELECT again. The table now shows eight rows, with Johnny De Goat appearing at user_id = 8, exactly as expected from the AUTO_INCREMENT behaviour.

```
select * from Users;
```

```

229 select * from Users; 334ms

```



user_id	name	phone_full	phone_masked	user_type	account_number
1	Ayo Mensah	+250781234567	+25078***567	personal	ACC200001
2	Joy Uwase	+250789876543	+25078***543	personal	ACC100002
3	Keza Mutoni	+250722345678	+25072***678	merchant	ACC100003
4	Claude Habimana	+250733456789	+25073***789	agent	ACC100004
5	Diane Ishimwe	+250756789012	+25075***012	personal	ACC100005
6	Eric Nkurunziza	+250768901234	+25076***234	personal	ACC100006
7	Holi Cow	+211923030987	+21192***987	personal	ACC100007
8	Johnny De Goat	+211923030674	+21192***674	personal	ACC100008

Query 3: UPDATE (modify an existing user)

Now we update the newly inserted user's name from "Johnny De Goat" to "Johnny Mike", targeting the row by its user_id. The client returns AffectedRows: 1, confirming that exactly the intended row was modified and nothing else.

```
Update Users
```

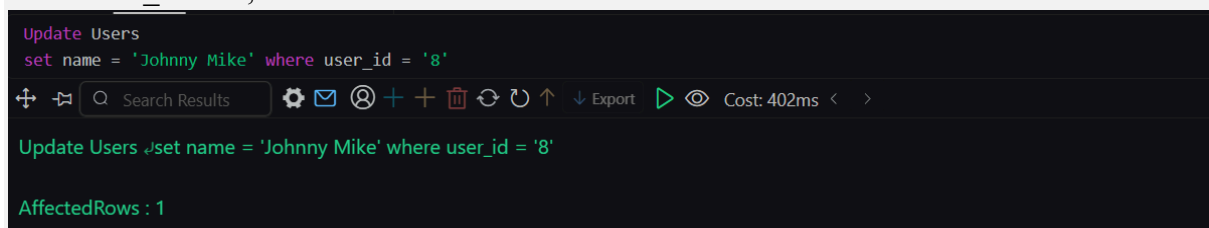
```
set name = 'Johnny Mike'
```

```
where user_id = '8';
```

```

Update Users
set name = 'Johnny Mike' where user_id = '8'

```



A follow-up SELECT confirms the change is reflected in the table. The row at user_id = 8 now shows "Johnny Mike", while the other columns (phone numbers, user_type, account number) remain untouched.

select * from Users LIMIT 100

<

Query 4: DELETE (remove a user)

Finally, we remove the test user we have been manipulating by issuing a DELETE statement scoped to user_id = 8. Again, AffectedRows: 1 confirms that exactly one row was removed.

Delete from Users
where user_id = '8';

Delete from Users
where user_id = '8'

Delete from Users where user_id = '8'

AffectedRows : 1

A final SELECT confirms the table is back to its original seven-row state. This complete CRUD cycle — Create, Read, Update, Delete — demonstrates that the schema is fully operational, that AUTO_INCREMENT behaves as expected, and that targeted modifications and deletions can be performed safely.

select * from Users LIMIT 100

<

9. JSON Data Modeling

The relational schema needs to be serializable into JSON for two use cases: the ETL output and the future dashboard API. Below are the JSON shapes that mirror each table, and one complete transaction object that nests all related information for an API response.

Entity-level JSON

Each entity below maps one-to-one with a row in its SQL table. Foreign keys are kept as IDs in this view, exactly as they exist in the database.

```

{
  "transaction_category": {
    "category_id": 1,
    "name": "P2P Transfer",
    "type": "debit",
    "direction": "OUT"
  },
  "user": {
    "user_id": 1,
    "name": "Ayo Mensah",
    "phone_full": "+250781234567",
    "phone_masked": "+25078***567",
    "user_type": "personal",
    "account_number": "ACC100001"
  },
  "transaction": {
    "transaction_id": 1,
    "transaction_date": 1714500000,
    "occurred_at": "2024-04-30 10:00:00",
    "amount": 5000.00,
    "fee": 50.00,
    "new_balance": 45000.00,
    "category_id": 1,
    "sender_message": "Lunch money",
    "token": "TKN-0001",
    "external_txn_id": "EXT-AA001",
    "body_raw": "TxId AA001 confirmed. RWF 5000 sent to Joy Uwase."
  },
  "transaction_users": {
    "transaction_id": 1,
    "user_id": 1,
    "role": "sender"
  },
  "system_log": {
    "log_id": 1,
    "timestamp": "2024-04-30 10:00:05",
    "level": "INFO",
    "message": "Transaction parsed successfully.",
    "record_reference": "TKN-0001",
    "transaction_id": 1
  },
  "system_message": {
    "received_at": 1714499990,
    "otp_code": null,
    "full_text": "TxId AA001 confirmed. RWF 5000 sent to Joy Uwase. New balance: RWF 45000.",
    "user_id": 1
  }
}

```

Completed transaction (nested API response)

This is the shape the dashboard API will return for a single transaction. Instead of leaving foreign keys as bare IDs, the related category, sender, receiver, and most recent log are nested inline so that one HTTP request returns everything the UI needs to render a transaction card.

```
{
  "Completed_transaction": {
    "transaction_id": 1,
    "occurred_at": "2024-04-30 10:00:00",
    "amount": 5000.00,
    "fee": 50.00,
    "new_balance": 45000.00,
    "token": "TKN-0001",
    "sender_message": "Lunch money",
    "category": {
      "category_id": 1,
      "name": "P2P Transfer",
      "type": "debit",
      "direction": "OUT"
    },
    "sender": {
      "user_id": 1,
      "name": "Ayo Mensah",
      "phone_masked": "+25078***567",
      "user_type": "personal"
    },
    "receiver": {
      "user_id": 2,
      "name": "Joy Uwase",
      "phone_masked": "+25078***543",
      "user_type": "personal"
    },
    "log": {
      "log_id": 1,
      "level": "INFO",
      "message": "Transaction parsed successfully.",
      "timestamp": "2024-04-30 10:00:05"
    }
  }
}
```

SQL ↔ JSON Mapping

The mapping between relational rows and the nested JSON above follows clear rules:

SQL Source	JSON Path	How it maps
transactions row	Completed_transaction.*	Scalar columns become top-level fields.

SQL Source	JSON Path	How it maps
transaction_categories row	Completed_transaction.category	Joined via transactions.category_id and nested as an object.
users row (role='sender')	Completed_transaction.sender	Resolved by joining transaction_users where role='sender'. Only phone_masked is exposed.
users row (role='receiver')	Completed_transaction.receiver	Same join pattern as sender, with role='receiver'.
system_logs row	Completed_transaction.log	The latest log entry for the transaction is nested as a single object.

Notice that phone_full is never serialized into the API response. Only phone_masked is sent. This is part of the security rules described in section 7.

10. Repository and Project Tracking

All technical artifacts are hosted in the team GitHub repository. The structure follows the assignment requirements:

- /docs — ERD diagram (erd_diagram.png) and supporting documentation.
- /database — SQL setup script (database_setup.sql).
- /examples — JSON examples for each entity and the nested completed_transaction.
- README.md — explains the database design, how to run the setup script, and how to load sample data.

Each team member's contribution is visible through individual commits in the repository's history. The Scrum board has been updated with completed Week 1 tasks and new Week 2 sprint tasks, and the link is provided in the submission form.

11. AI Usage Log

In line with the course AI usage policy, this section logs every interaction with AI tools during this assignment.

Date	Tool	Purpose	How the output was used
Week 2	Claude (Anthropic)	Grammar and syntax checking of the reflection text written by the team.	AI suggestions were reviewed line by line; only spelling, punctuation, and sentence-flow fixes were accepted. The technical design decisions are entirely the team's own.

Date	Tool	Purpose	How the output was used
Week 2	Claude (Anthropic)	Formatting the team's notes into the standard sections required by the rubric (data dictionary, security rules, and sample queries section).	AI was used as a layout assistant only. Every table entry, constraint, and query was authored by the team based on our ERD; AI did not invent entities, columns, or relationships.
Week 2	MySQL official documentation	Verifying syntax for CHECK constraints and ENUM types.	Used as a reference, not for generating logic. Citation kept in the AI usage section of the repo.
Week 2	Claude	This was used to generate sample data for our table so we can easily perform the CRUD operation on it.	This was used to test the table accuracy, and if our sql operation are flawless, which we were able to do successfully.

No AI tool was used to design the ERD, write business logic, decide on entities or relationships, or generate the reflection content. All design decisions, query writing, and explanations were produced by the team.

12. Conclusion

The schema captures every transaction type identified in the MoMo SMS dataset, handles the irregular field availability between transaction types using thoughtful NULL rules, isolates OTP messages into their own entity to keep the financial dataset clean, and resolves the natural many-to-many relationship between transactions and users with a junction table that also stores each user's role. The sample queries demonstrate that the design is ready to power a dashboard, and the security rules add a layer of protection for both data quality and user privacy. The next steps will be to connect the ETL pipeline, so live data populates the schema, and then to build the dashboard against the API shape described in section 9.